OS-ORANGE PROBLEM-2

NAME-VANISHREE

SRN:PES2UG24CS665

Phase 1: Object Storage

Screenshot 1A: Object Store Test Output

Description:

This screenshot shows the successful execution of the object store test program (test_objects). All tests passed, confirming correct implementation of object writing, reading, deduplication, and integrity verification.

```
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
```

testcases ⊗ 2 ⚠ 0 Ln 603, Col 1 (23419 selected)

Screenshot 1B: Object Storage Directory Structure

Description:

This screenshot shows the internal structure of the object store inside the .pes/objects directory. The objects are stored using content-addressable storage and are organized into subdirectories based on the first two characters of their SHA-256 hash, demonstrating directory sharding.

```
All Phase 1 tests passed.
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
Run Testcases  2 0                                   Ln 603, Col 1 (23419 selec
```

Phase 2: Tree Objects

Screenshot 2A: Tree Serialization Test Output

Description:

This screenshot shows the successful execution of the tree test program (test_tree). It verifies correct serialization and parsing of tree objects, ensuring that directory structures are stored and reconstructed accurately.

```
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
un Testcases  2 0                                   Ln 603, Col 1 (23419 selected)
```

Screenshot 2B: Raw Tree Object (Hex Dump)

Description:

This screenshot shows the raw binary representation of a tree object using xxd. It demonstrates how directory entries are stored in the format "<mode> <name>\0<hash>", confirming correct tree object serialization.

```
Committed: 82ffd0a4d509... temp commit
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/82/ffd0a4d5096c3bc273da90c742c8bb0c20d9515c9ecf76b485d351e7ebd0ed
.pes/objects/d1/f8c0e987c2dbf3ffc0c9030c3da3f11bd85f0db2c2955568743b576468d9a9
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34
00000030: 9025 de57 6db9 ebb4                      .%.Wm...
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ xxd .pes/objects/d1/f8c0e987c2dbf3ffc
0c9030c3da3f11bd85f0db2c2955568743b576468d9a9 | head -20
00000000: 7472 6565 2034 3800 3130 3036 3634 2066  tree 48.100664 f
00000010: 696c 652e 7478 7400 2cf8 d83d 9ee2 9543  ile.txt.,..=...C
00000020: b34a 8772 7421 fdec b7e3 f3a1 83d3 3763  .J.rt!........7c
00000030: 9025 de57 6db9 ebb4                      .%.Wm...
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
```

Phase 3: Index (Staging Area)

Screenshot 3A: Staging and Status Output

Description:
This screenshot shows the workflow of initializing the repository, adding files to the staging area, and checking the status. It confirms that files are correctly staged and reflected in the index.

```
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ echo "hello" > file1.txt
echo "world" > file2.txt
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ ./pes add file1.txt file2.txt
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ ./pes status
Staged changes:
  staged:     file.txt
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  tree.c
  untracked:  test_objects.c
  untracked:  commit.h
  untracked:  PES2UG24CS665-pes-vcs-report.pdf
  untracked:  README.md
  untracked:  pes.c
  untracked:  index.c
  untracked:  tree.h
  untracked:  Makefile
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  pes.h
  untracked:  commit.c
  untracked:  test_tree.c
  untracked:  .gitignore
```

Phase 3: Index (Staging Area)

Screenshot 3B: Index File Content

Description:
This screenshot shows the contents of the .pes/index file. It demonstrates how staged files are recorded with their mode, hash, modification time, size, and file path.

```
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776808797 6 file.txt
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776808867 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776808867 6 file2.txt
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
Ln 276, Col 26
```

Phase 4: Commits and History

Screenshot 4A: Commit Log Output

Description:

This screenshot shows the commit history generated using pes log. It confirms that multiple commits are correctly created, linked using parent references, and displayed in reverse chronological order with author, timestamp, and message.

```
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776785091 6 file2.txt
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ ./pes commit -m "first commit"
./pes log
Committed: 932eaba5c1db... first commit
commit 932eaba5c1db51e495c6efe7225d5a513a3b6b61b4b9afe1764e810636ed0cff
Author: vanishree <PES2UG24CS665>
Date:   1776785354

    first commit

vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ echo "new" >> file1.txt
./pes add file1.txt
./pes commit -m "second commit"
./pes log
Committed: 7e689e0910aa... second commit
commit 7e689e0910aa29a93f5020b0509b5b3cf85ac7df7150cf7e175c04b24204e4f9
Author: vanishree <PES2UG24CS665>
Date:   1776785362

    second commit

commit 932eaba5c1db51e495c6efe7225d5a513a3b6b61b4b9afe1764e810636ed0cff
Author: vanishree <PES2UG24CS665>
Date:   1776785354

    first commit

vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
Testcases  ⊗ 2 ⚠ 0                                  Ln 603, Col 1 (23419 selected)
```

Screenshot 4B: Object Store and Repository Structure

Description:

This screenshot shows all files inside the .pes directory, including objects, references,

HEAD, and index. It demonstrates how commits, trees, and blobs are stored in the object store and how repository structure grows over time.

```
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/18/46c583367706bbc87cd9a4899fbc35c49c1c35a08da2f014b6378f991daba3
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/7e/689e0910aa29a93f5020b0509b5b3cf85ac7df7150cf7e175c04b24204e4f9
.pes/objects/93/2eaba5c1db51e495c6efe7225d5a513a3b6b61b4b9afe1764e810636ed0cff
.pes/objects/9d/ccfa78f4be6ba6b867c9ffc26b42b4debb7444452d16f0106d2cbd0724741e
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e3/897b13c40ac3acc9f37fd01ca1a2ab88fcca9adf8d459b6c5c1f9c745aaf44
.pes/refs/heads/main
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
Testcases  2 0                                   Ln 603, Col 1 (23419 selected)
```

Screenshot 4C: HEAD and Branch Reference

Description:

This screenshot shows the contents of the HEAD file and the current branch reference. The HEAD file points to refs/heads/main, indicating the active branch, while the branch file stores the latest commit hash. This confirms that the branch pointer is correctly updated after each commit.

```
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
7e689e0910aa29a93f5020b0509b5b3cf85ac7df7150cf7e175c04b24204e4f9
ref: refs/heads/main
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
Testcases  2 0                                   Ln 603, Col 1 (23419 s
```

Final: Integration Test Execution

Description:

This screenshot shows the successful execution of the complete PES-VCS integration test. It verifies repository initialization, file staging, multiple commits, commit history traversal, reference updates, and object store creation. All components work together correctly, confirming a fully functional version control system.

```
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 6aed3d5afa91... Initial commit

Log after first commit:
commit 6aed3d5afa911f9c9bbb389eb9c864fe512b1607bc0858aef7b7d8fa0b4c1c46
Author: vanishree <PES2UG24CS665>
Date:   1776785736

    Initial commit


--- Second Commit ---
Committed: 92b044345135... Update file.txt

--- Third Commit ---
Committed: f7077a83d1c5... Add farewell
```

```
--- Full History ---
commit f7077a83d1c522a91f4e298746b607f034deb2545a0063251ca02425e5f97bd5
Author: vanishree <PES2UG24CS665>
Date:   1776785736

    Add farewell

commit 92b044345135c8ae4247d2f43359cc2ca34704f22dd3a9e164bf6a0aa71cab20
Author: vanishree <PES2UG24CS665>
Date:   1776785736

    Update file.txt

commit 6aed3d5afa911f9c9bbb389eb9c864fe512b1607bc0858aef7b7d8fa0b4c1c46
Author: vanishree <PES2UG24CS665>
Date:   1776785736

    Initial committial comQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f7077a83d1c522a91f4e298746b607f034deb2545a0063251ca02425e5f97bd5

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6a/ed3d5afa911f9c9bbb389eb9c864fe512b1607bc0858aef7b7d8fa0b4c1c46
.pes/objects/84/c5fed72c2dd9a3130e2e9b98033827f62793fb80d0295ccedf5fc7972d96eb
.pes/objects/92/b044345135c8ae4247d2f43359cc2ca34704f22dd3a9e164bf6a0aa71cab20
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/de/34fb483a5c16ce29ec3195dd505dfda7e35cc5fe16a99da4b2455b332a93cf
.pes/objects/ec/0af37b159ae92a5850ceaa6e497860729d6158aab2c6833b6068a1e3c5e8ef
.pes/objects/f7/077a83d1c522a91f4e298746b607f034deb2545a0063251ca02425e5f97bd5

=== All integration tests completed ===
vaniprsy@vaniprsy-IdeaPad-Slim-3-15IRH10:~/PES2UG24CS665-pes-vcs$
```

Phase 5: Branching and Checkout

Q5.1: How would you implement pes checkout <branch>?

A branch in PES-VCS is a file inside .pes/refs/heads/ that stores a commit hash. The HEAD file points to the current branch.

To implement pes checkout <branch>, the following steps are required:

1. Verify that the branch exists inside .pes/refs/heads/.
2. Update the HEAD file to point to the new branch (e.g., ref: refs/heads/<branch>).

3. Read the commit hash stored in the branch file.
4. Load the corresponding commit object using object_read.
5. Retrieve the root tree from the commit.
6. Recursively reconstruct the working directory by:
    a. Reading tree objects
    b. Creating directories and files
    c. Writing blob contents to files

This operation is complex because the working directory must exactly match the target commit. Any differences must be resolved carefully, especially if files already exist or have been modified.

Q5.2: How to detect a dirty working directory before checkout?

A dirty working directory means there are uncommitted changes that could be overwritten.

To detect this:

1. Compare each file in the working directory with the corresponding entry in the index.
2. For each tracked file:
    a. Check file size and modification time
    b. Recompute the hash and compare with the index hash
3. If any file differs from the index, it is considered modified.
4. Also compare index entries with the current HEAD commit to detect staged but uncommitted changes.

If any differences are found, checkout should be aborted to prevent data loss.

Q5.3: What is detached HEAD and what happens if you commit?

A detached HEAD occurs when the HEAD file directly contains a commit hash instead of pointing to a branch.

In this state:

- New commits are created normally
- However, no branch reference is updated
- These commits are not reachable from any branch

As a result, these commits can be lost if no reference points to them.

To recover:

- Create a new branch pointing to that commit
- Example: manually write the commit hash into .pes/refs/heads/<new-branch>

Phase 6: Garbage Collection and Space Reclamation

Q6.1: How to find and delete unreachable objects?

Garbage collection removes objects that are no longer reachable from any branch.

Algorithm:

1. Start from all branch references in .pes/refs/heads/.
2. For each commit:
    a. Mark the commit as reachable
    b. Recursively follow:
        i. Parent commits
        ii. Tree objects
        iii. Blob objects
3. Store all reachable hashes in a data structure such as a hash set.
4. Traverse .pes/objects/ directory:
    a. If an object's hash is not in the reachable set, delete it

For a repository with 100,000 commits and 50 branches:

- Each commit, tree, and blob must be visited once
- Total objects could be several hundred thousand depending on project size

Q6.2: Why is garbage collection dangerous during commits?

Garbage collection can interfere with a commit if both run simultaneously.

Race condition:

1. A commit is being created but not yet referenced by a branch.
2. Garbage collection runs at the same time.
3. Since the new commit is not yet reachable, GC considers it unused.
4. GC deletes the object.
5. The commit process then tries to reference a deleted object, causing corruption.

To avoid this, real systems like Git:

- Use locking mechanisms to prevent concurrent operations
- Delay deletion of recent objects
- Use safe temporary states before making objects permanen